CSC3100: Stack and Queue

Tianshu Yu

2024 Fall

Midterm

- 2023.11.5 (Sun.). 13:30 – 15:30
- Sports Hall

What is a Stack

- Stack of Books



Motivating examples

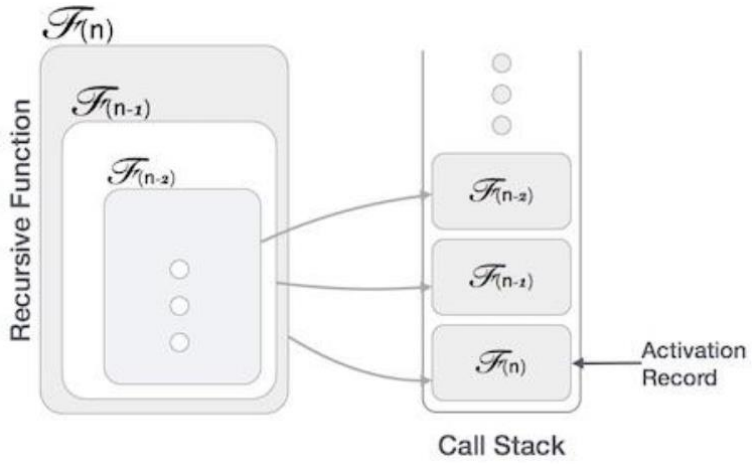
- Plates and bowls





- Program codes
  - Call functions, e.g., recursive functions

```
public class test {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello ");
        System.out.println("before change, sb is "+sb.toString());
        change(sb);
        System.out.println("after change, sb is "+sb.toString());
    }
    public static void change(StringBuffer stringBuffer){
        stringBuffer = new StringBuffer("Hi ");
        stringBuffer.append("world !");
    }
}
```



Stack

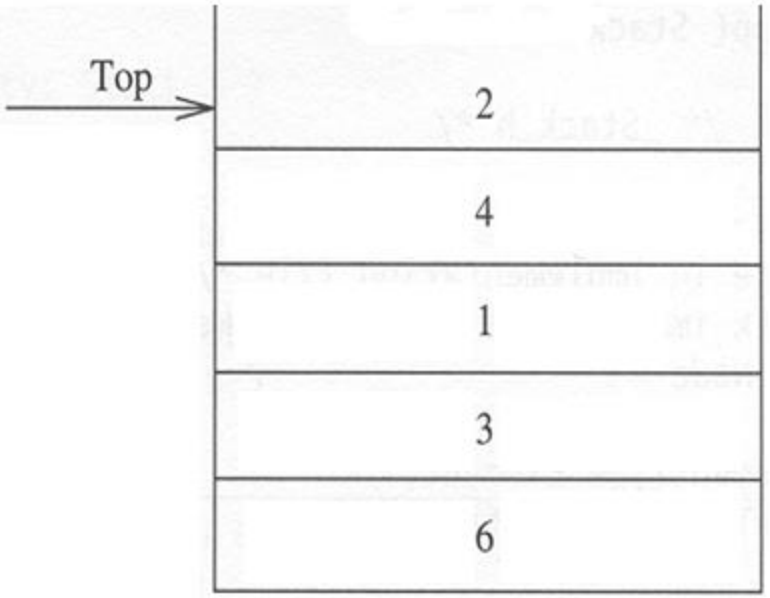
- Stack model



Stack

- LIFO structure (last in first out)
- Homogeneous elements
- Access from the top
- Basic operations
  - pop ()
  - push (i)

  - makeEmpty ()
  - top ()
  - isEmpty()



- Implementations
  - Linked list
  - Arrays

Stacks

- A stack is sometimes also called a pushdown store.
- What are some applications of stacks?
  - Program execution
  - Parsing
  - Evaluating postfix expressions

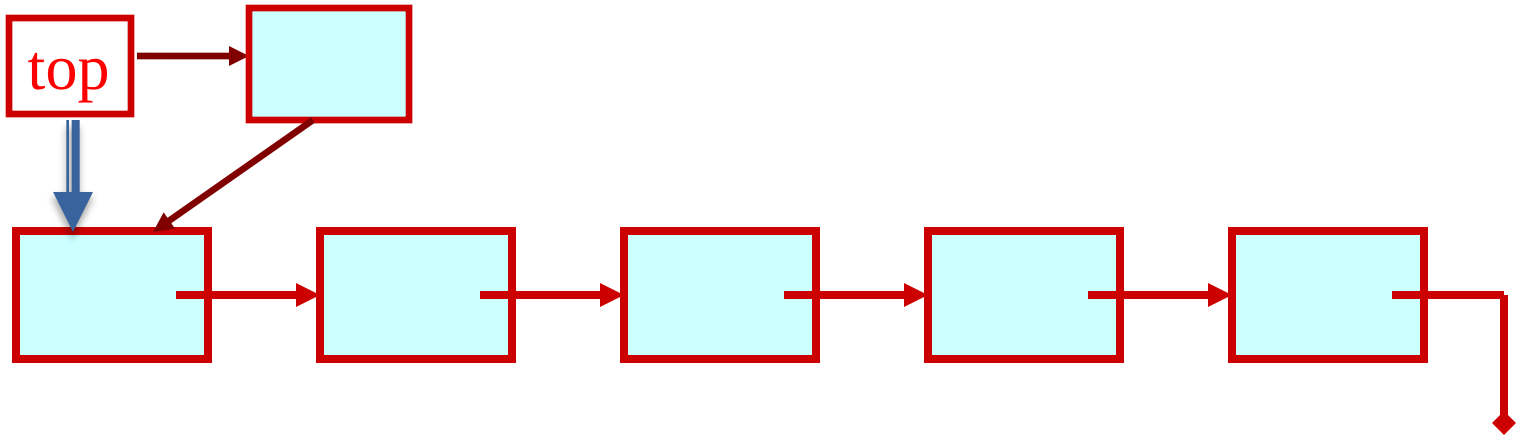


Stacks

- Problem:
  - What happens if we try to pop an item off the stack when the stack is empty?
    - This is called a stack underflow. The pop method needs some way of telling us that this has happened. You are also required to know how to realize such functionality using Java.

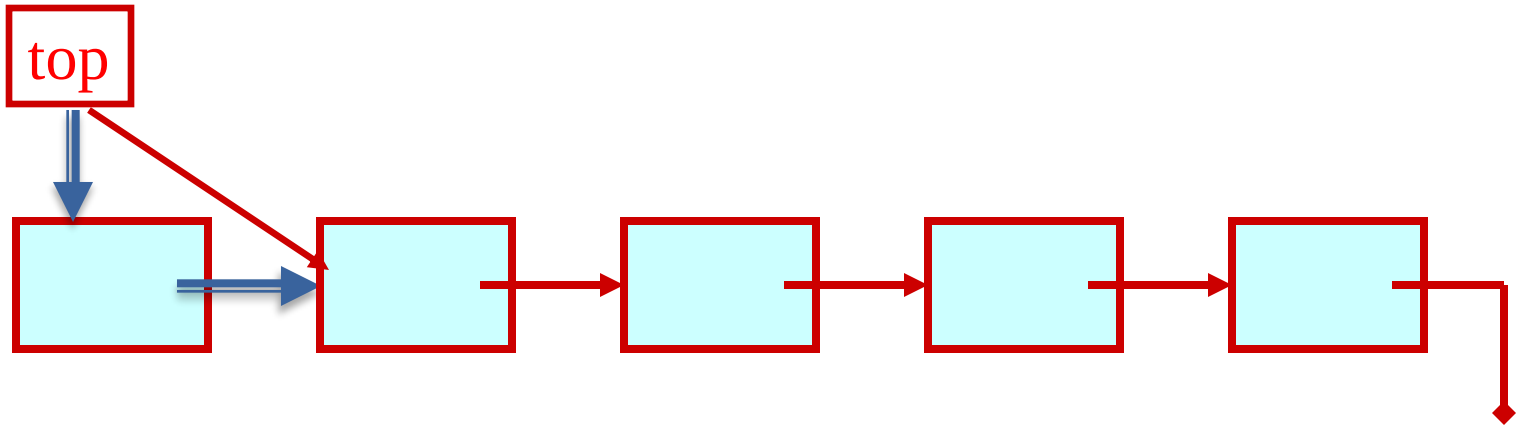
Push using Linked List

PUSH OPERATION



Pop using Linked List

POP OPERATION



Implementation of Stack using Linked List

```
class Node {
        Node next;
        Object element;
}

class Stack {
        Node next;
}
```

Implementation of Stack using Linked List

- Create an empty stack

```
public Stack() {                    //constructor
        this.next = null;
}
```

- Test for an empty stack

```
public boolean isEmpty( ) {
        return (this.next == null)
}
```

Implementation of Stack using Linked List

- Make it empty

```
void makeEmpty() {
        while(this.next != null){
                Node tmpNode = this.next;
                this.next = this.next.next;
                free(tmpNode);
        }
}
```

- Push onto a stack

```
void push(Object x) {
                Node tmpNode = new Node();
                tmpNode.element = x;
                tmpNode.next = this.next;
                this.next = tmpNode;
}
```

Implementation of Stack using Linked List

- Return top element in a stack

```
public Object top( ) {
        if (!isEmpty())
                return next.element;
        else {
                return null;
        }
}
```

- Pop from a stack

```
public Object pop() {
  Node firstNode = null;
  if (isEmpty()) {
          return null;
  } else {
          firstNode = this.next;
          this.next = firstNode.next;
          return firstNode.element;
  }
}
```

Implementation of Stack using Array

```
class Stack {
        final static int MIN_STACK_SIZE = 5;
        int topOfStack = -1;
        Object[ ] array;
}
```

Implementation of Stack using Array

- Stack creation

```
public Stack (int maxElements) {
   int capacity = maxElements;

   if (maxElements < MIN_STACK_SIZE)
         capacity = MIN_STACK_SIZE;

   array = new Object[capacity];
}
```

Implementation of Stack using Array

- Test for full stack

```
public boolean isFull() {
        return (topOfStack == array.length - 1);
}
```

- Push an element onto the stack

```
public boolean push(Object x) {
        if (isFull())
                return false;
        else
                array[++topOfStack] = x;
        return true;
}
```

Implementation of Stack using Array

- Return top of stack

```
public Object top() {
        if (!isEmpty())
                return array[topOfStack];
        else
                return null;
}
```

- Pop element from stack

```
public Object pop() {
        if (!isEmpty())
                return array[topOfStack--];
        else
                return null;
}
```

Comparison of these two implementation

- Using list saves space

- Using array is faster. Why?
  - Two reasons:
    - Memory allocation
    - Continuous memory can be loaded into cache

- A lot to learn for implementing a fast program

Parentheses Matching

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

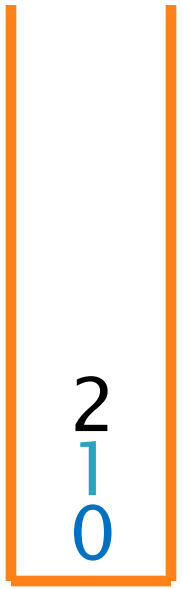
- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)
  - Output pairs (u,v) such that the left parenthesis at position u is matched with the right parenthesis at v.
    - (2,6) (1,13) (15,19) (21,25) (27,31) (0,32) (34,38)
- (a+b))*((c+d)
  - (0,4)
  - right parenthesis at 5 has no matching left parenthesis
  - (8,12)
  - left parenthesis at 7 has no matching right parenthesis

Example: Parentheses Matching

- scan expression from left to right
- when a left parenthesis is encountered, add its position to the stack
- when a right parenthesis is encountered, remove matching position from stack

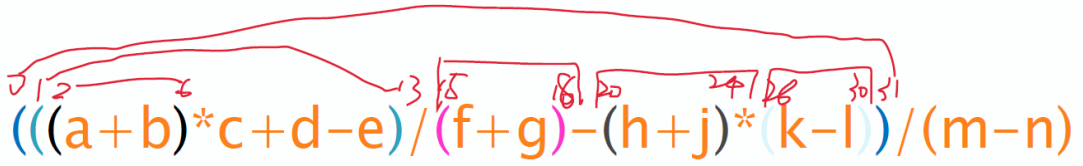
Example

- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



Example

- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



(2,6) (1,13)

Example

- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



(2,6) (1,13)(15,19)

Example

- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



(2,6) (1,13)(15,19)(21,25)

Example

- (((a+b)*c+d-e)/(f+g)-(h+j)*(k-l))/(m-n)



(2,6) (1,13)(15,19)(21,25)(27,31)(0,32)

- and so on

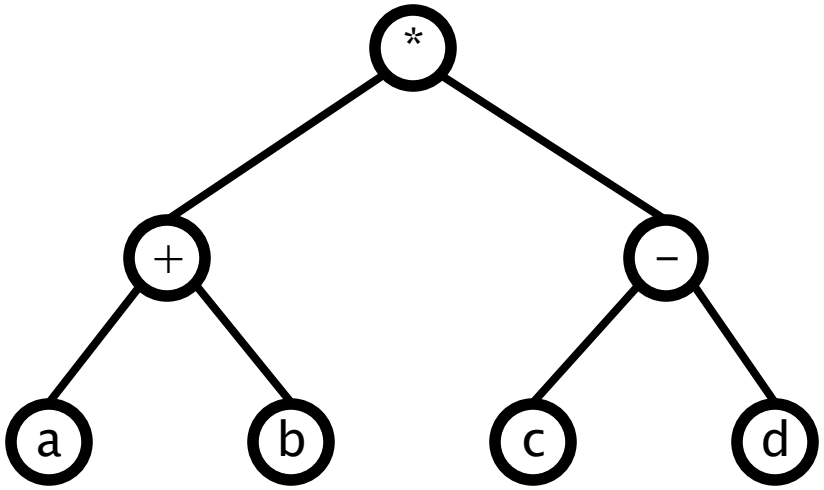
Example: Prefix to Postfix

- Prefix expression vs. postfix expression

  Prefix: *+ab-cd

  Postfix: ab+cd-*

- Parsing tree:



Example

- Algorithm:
  - Read the Prefix expression in reverse order (from right to left)
  - If the symbol is an operand, then push it onto the Stack
  - If the symbol is an operator, then pop two operands from the Stack. Create a string by concatenating the two operands and the operator after them.
    string = operand1 + operand2 + operator
    And push the resultant string back to Stack
  - Repeat the above steps until end of Prefix expression.

```java
// JavaProgram to convert prefix to postfix
import java.util.*;

class GFG {

    // function to check if character
    // is operator or not
    static boolean isOperator(char x)
    {
        switch (x) {
        case '+':
        case '-':
        case '/':
        case '*':
            return true;
        }
        return false;
    }

    // Convert prefix to Postfix expression
    static String preToPost(String pre_exp)
    {

        Stack<String> s = new Stack<String>();

        // length of expression
        int length = pre_exp.length();
```

```
        // reading from right to left
        for (int i = length - 1; i >= 0; i--)
        {
            // check if symbol is operator
            if (isOperator(pre_exp.charAt(i)))
            {
                // pop two operands from stack
                String op1 = s.peek();
                s.pop();
                String op2 = s.peek();
                s.pop();

                // concat the operands and operator
                String temp = op1 + op2 + pre_exp.charAt(i);

                // Push String temp back to stack
                s.push(temp);
            }

            // if symbol is an operand
            else {
                // push the operand to the stack
                s.push(pre_exp.charAt(i) + "");
            }
        }

        // stack contains only the Postfix expression
        return s.peek();
    }
```

```
    // Driver Code
    public static void main(String args[])
    {
        String pre_exp = "*-A/BC-/AKL";
        System.out.println("Postfix : "
                           + preToPost(pre_exp));
    }
}
```

Output

```
Postfix : ABC/-AK/L-*
```

Consider

- How to **generate** all valid expressions consisting of *n* parentheses?

Example: next greater element

- Given an array, print the Next Greater Element (NGE) for every element. Elements for which no greater element exist, consider the next greater element as -1.

  Input: [4, 5, 2, 25]

  Output: 4 → 5

  5 → 25

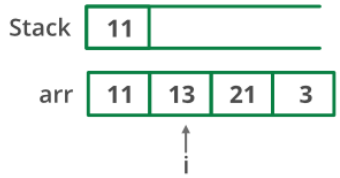
  2 → 25

  25 → -1

Example: next greater element

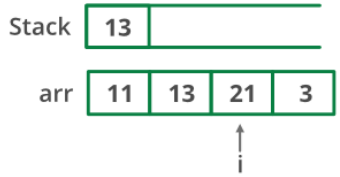
- Algorithm
  - Push the first element to stack.
  - Pick rest of the elements one by one and follow the following steps in loop.
    - 1. Mark the current element as *next*.
    - 2. If stack is not empty, compare top element of stack with *next*.
    - 3. If next is greater than the top element, Pop element from stack. *next* is the next greater element for the popped element.
    - 4. Keep popping from the stack while the popped element is smaller than *next*. *next* becomes the next greater element for all such popped elements.
  - Finally, push the next in the stack.
  - After the loop in step 2 is over, pop all the elements from the stack and print -1 as the next element for them.

Initially :



Step 1:

arr[i] > S.top
S.pop()
Next greater element of 11 is 13
S.push(arr[i])



Step 2:

arr[i] > S.top()
S.pop()
Next greater element of 13 is 21
S.push(arr[i])



Step 3:

arr[i] < S.top()
S.push(arr[i])



i reached to end of the array

Step 4:

For all elements in stack next greater element is
Next greater element of 3 is -1
S.pop()

Step 5:

Next greater element of 21 is -1
S.pop()

What is a Queue?



Real-world queues

- A queue of customers waiting to buy a product from a shop, where the customer that came first is served first

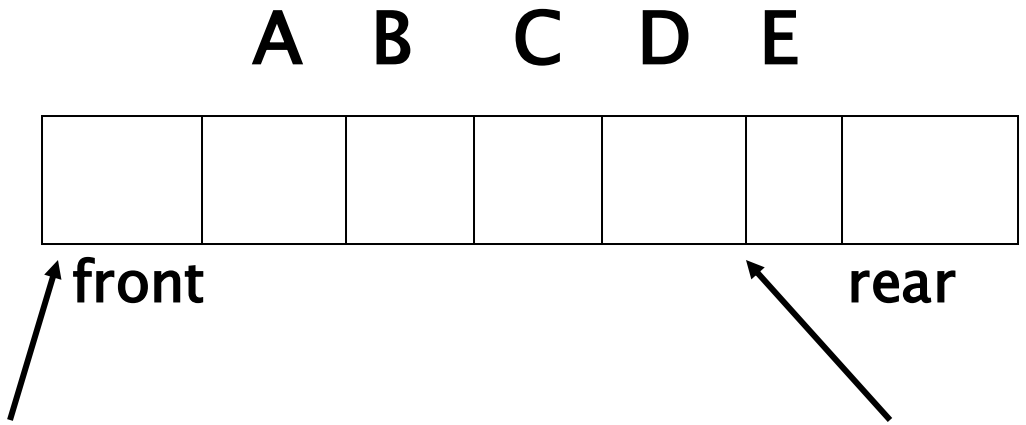


- A queue of students waiting to take a bus at a bus station, where the student that came first gets on first



Queue

- Ordered collections of data items
  - Delete item at front of the queue
  - Insert item at rear of the queue
  - A FIFO (first-in first-out) structure



Queue applications in CS

- Used as buffers MP3 player, CD player, etc.
  - Used to maintain the play list in media players in order to add and remove the songs from the play-list



- Used in OS
  - Multiple data transformation tasks
  - Scheduling shared resources like CPU and GPU



- Suppose we have a printer shared by several users, and it can serve a single request at a time
  - When any print request comes, and if the printer is busy, the request will be put into a queue
  - If the requests are available in the queue, the printer takes a request from the queue and serves it



Queue

- Basic operations

(1) enqueue (ElementType X)

which inserts an element at the end of the list (called the rear)

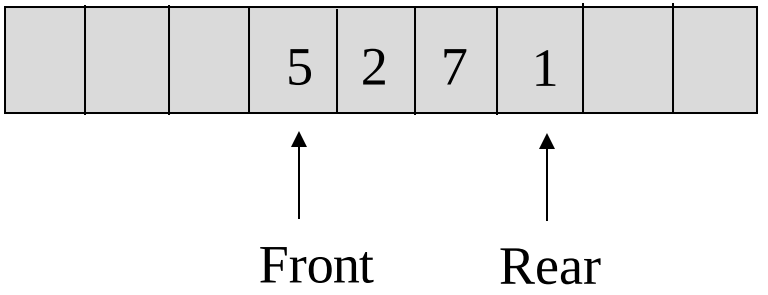
(2) dequeue()

which deletes (and returns) the element at the start of the list (known as the front)

Queue using array

- Array implementation

```
public class Queue_Array {
        int capacity;
        int front, rear, size;
        ElementType[] array;
}
```



Operations:

- To enqueue an element X

  Increment Size and Rear, then set
  Queue[Rear] = X;

- To dequeue an element

  Return the value of Queue[Front], decrease Size, and then increment Front.

Implementing a Queue

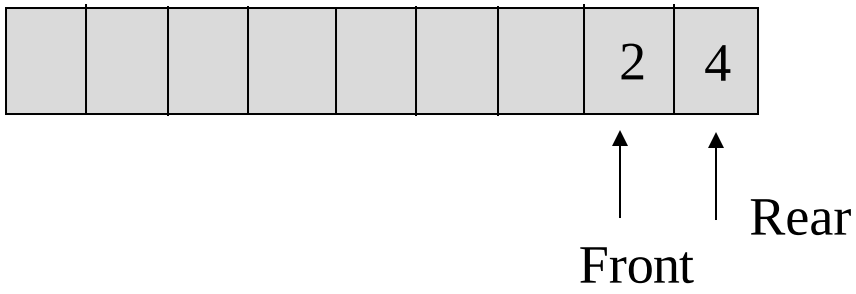
- There are two options for implementing a queue using an array:
- Option 1:
  - *Enqueue* at data[0] and shift all of the rest of the items in the array down to make room.
  - *Dequeue* from data[numItems-1]

Implementing a Queue

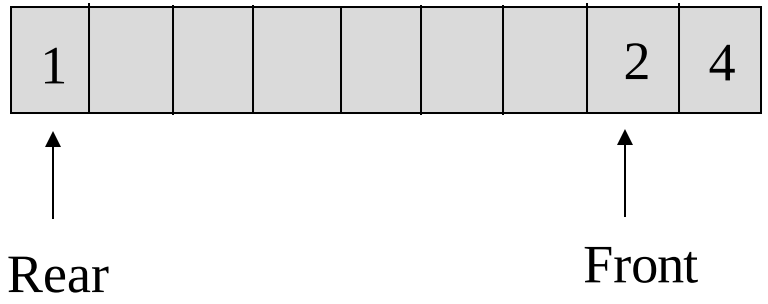
- Option 2
  - *Enqueue* at data[rear+1]
  - *Dequeue* at data[front]
  - The rear variable always contains the index of the last item in the queue.
  - The front variable always contains the index of the first item in the queue.
  - When we reach the end of the array, wrap around to the front again.

Circular Array Implementation

Initial State

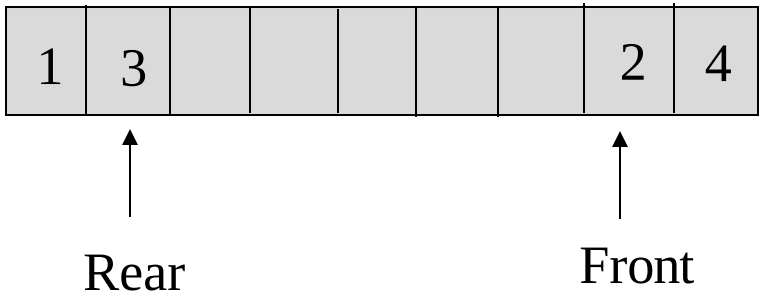


After enqueue(1)

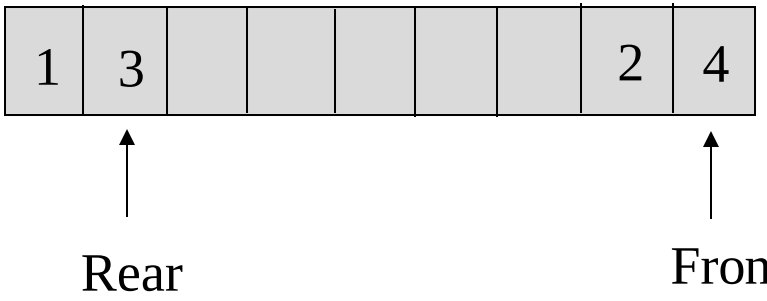


Circular Array Implementation

After enqueue(3)

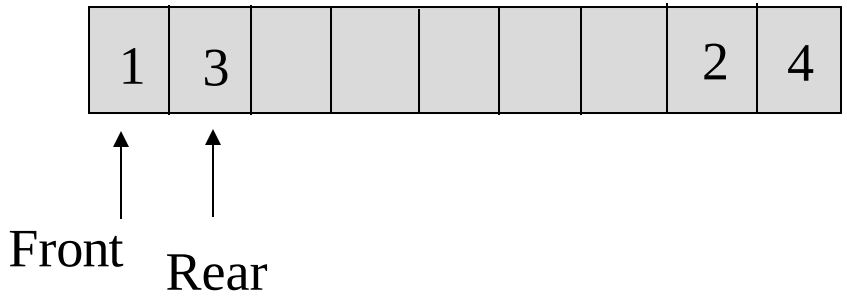


After dequeue, which returns 2

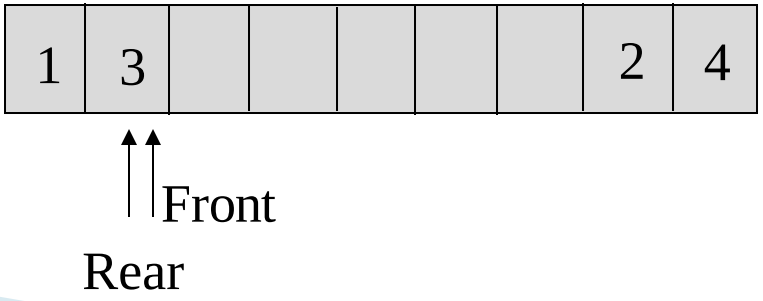


Circular Array Implementation

After dequeue, which returns 4



After dequeue, which returns 1



Queue implementation

- Create an empty queue

```
public void makeEmpty() {
        size = 0;
        front = 1; rear = 0;
        array = new Object[capacity];
}
```

Queue implementation

- Two checks

```
public boolean isEmpty() {
        return size == 0;
}

public boolean isFull() {
        return size == capacity;
}
```

Queue implementation

```
public boolean enqueue(Object x) {
        if (isFull())
                return false;
        else {
                size++;
                rear = (rear + 1) % capacity;
                array[rear] = x;
                return true;
        }
}
```

Queue implementation

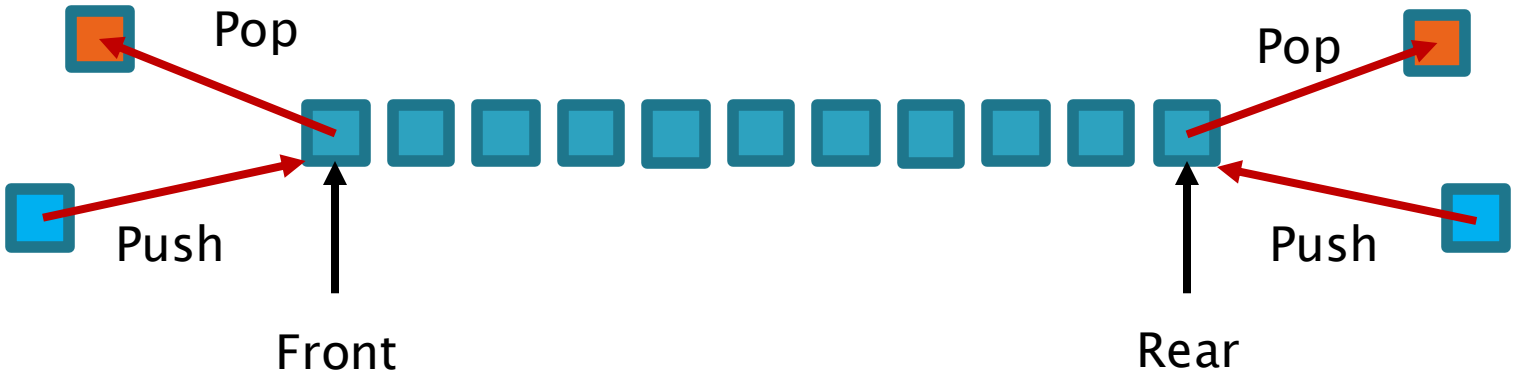
```
public ElementType dequeue() {
        int first;
        if (isEmpty())
                return null;
        else {
                size--;
                first = front;
                front = (front + 1) % capacity;
                return array[first];
        }
}
```

Question

- How to implement queue using list?

Double-ended Queue

- A Deque or Deck is a double-ended queue
- Allows elements to be added or removed on either ends



Applications of Queues

- Print jobs
  - Jobs are arranged in order of arrival.
- Computer networks
  - Users are given access to the file server on a first-come first-served basis.
- Real-life waiting lines
- Round-robin scheduling in processors

Example

- Given an array and a positive integer k, find the first negative integer for each window(contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window.

Input : arr[] = {-8, 2, 3, -6, 10}, k = 2

Output : -8 0 -6 -6

First negative integer for each window of size k

{-8, 2} = -8

{2, 3} = 0 (does not contain a negative integer)

{3, -6} = -6

{-6, 10} = -6

Example

- A naïve approach
  - Run two loops. In the outer loop, take all subarrays(windows) of size k. In the inner loop, get the first negative integer of the current subarray(window).
- Time complexity O(n*k)
- Is there a more efficient method?

Example

- Let's try to use queue
  - Create a deque Di, with capacity *k*.
  - Traverse the elements of the first window, if the element is negative, push its index to the end of the Di.
  - At the end of the traversal if the Di is empty print 0, else the first element in the Di corresponds to the index of first negative element.
  - Traverse in the remaining array(start from index k to n – 1), while the front of Di is less than (i – k + 1) remove it. If the current element is negative, add it to Di.
  - The first element in Di corresponds to the first negative element for this window, if Di is empty there is no negative element in this window, print 0.
- Time complexity O(n)

```
static void printFirstNegativeInteger(int arr[],
                                      int n, int k)
{
    // A Double Ended Queue, Di that will
    // store indexes of useful array elements
    // for the current window of size k.
    // The useful elements are all negative integers.
    LinkedList<Integer> Di = new LinkedList<>();

    // Process first k (or first window)
    // elements of array
    int i;
    for (i = 0; i < k; i++)

        // Add current element at the rear of Di
        // if it is a negative integer
        if (arr[i] < 0)
            Di.add(i);
```

```
    for ( ; i < n; i++)
    {
        // if Di is not empty then the element
        // at the front of the queue is the first
        // negative integer of the previous window
        if (!Di.isEmpty())
            System.out.print(arr[Di.peek()] + " ");

        // else the window does not have a
        // negative integer
        else
            System.out.print("0" + " ");

        // Remove the elements which are
        // out of this window
        while ((!Di.isEmpty()) &&
                Di.peek() < (i - k + 1))
            Di.remove(); // Remove from front of queue

        // Add current element at the rear of Di
        // if it is a negative integer
        if (arr[i] < 0)
            Di.add(i);
    }

    // Print the first negative
    // integer of last window
    if (!Di.isEmpty())
        System.out.print(arr[Di.peek()] + " ");
    else
        System.out.print("0" + " ");
}
```